

Learning Objectives

By the end of today, you can:

- Explain why linear models struggle with non-linear structure and how basis functions and learned features address this.
- Sketch a 1-hidden-layer MLP and a stacked MLP; write $h = g(b + Wx)$, track the shapes of W, b, h , and count parameters across layers.
- Choose suitable *hidden* activations (ReLU/LeakyReLU/SiLU/GELU) and *output* activations per task, pairing them with the correct losses.
- Describe the training loop (forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update) and the roles of initialization, normalization, and optimizers.
- Diagnose common training pathologies (vanishing/exploding gradients, dying ReLUs, over/underfitting) and apply regularization (weight decay, dropout, early stopping).
- Evaluate models with proper splits and metrics (MSE/MAE for regression; accuracy/F1 for classification) while avoiding data leakage.
- Implement and test a small network in the lab to reach a target metric, and justify your design choices.

Advantages of linear models

- In lectures 4 and 5, we saw models for regression and classification that consisted of **linear parametric functions** of the inputs:

- Linear regression

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_D x_D + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- Logistic regression

$$p(y_j = 1) = \frac{1}{1 + \exp(-\beta^T \mathbf{x}_j)}$$

Advantages of linear models

- In lectures 4 and 5, we saw models for regression and classification that consisted of **linear parametric functions** of the inputs:

- Linear regression

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_D x_D + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- Logistic regression

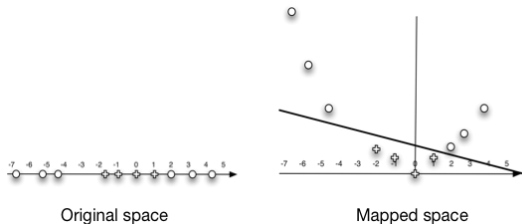
$$p(y_j = 1) = \frac{1}{1 + \exp(-\beta^T \mathbf{x}_j)}$$

- Useful analytical and computational properties!
 - Fixed number of parameters
 - Convex objective function
 - Computationally fast
 - Interpretability
- However, linear assumption can often be too strong!

Basis functions and curse of dimensionality

- Can be made more expressive by considering **fixed** basis functions $\phi(\mathbf{x})$
 - Recall the kernel trick...

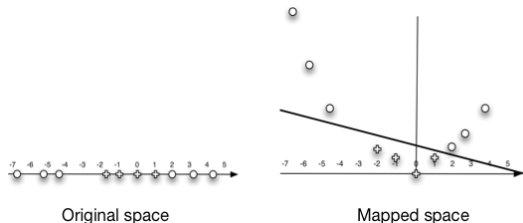
Example: $\phi(x) = (x, x^2)$



Basis functions and curse of dimensionality

- Can be made more expressive by considering **fixed** basis functions $\phi(\mathbf{x})$
 - Recall the kernel trick...

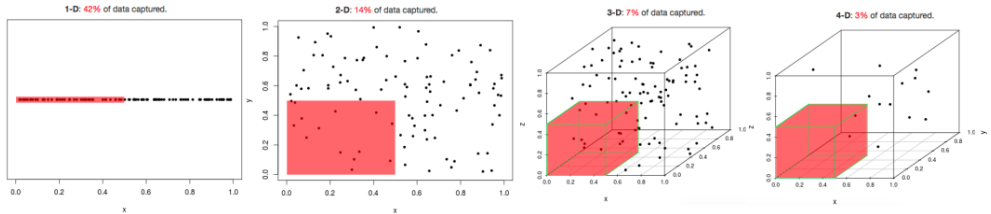
Example: $\phi(x) = (x, x^2)$



- But that quickly raises the number of features (input dimensionality) and corresponding parameters beyond manageable values!

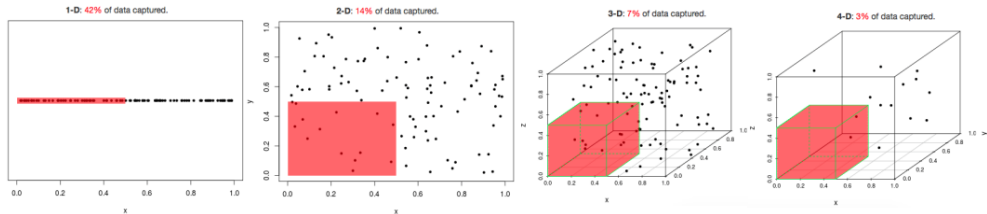
Basis functions and curse of dimensionality

- Curse of dimensionality: as dimensionality D increases, the volume of the space increases so fast that the available data becomes sparse



Basis functions and curse of dimensionality

- Curse of dimensionality: as dimensionality D increases, the volume of the space increases so fast that the available data becomes sparse



- Cannot just keep adding basis functions $\phi(\mathbf{x})$!
 - Data becomes sparser
 - Number of parameters increases
- To scale to large datasets, it is necessary to **adapt** the basis functions to the data

Coping with the curse of dimensionality

- SVM's approach:
 - Define basis functions that are centred on the training data points
 - Select only a subset of these during training - *support vectors*

Coping with the curse of dimensionality

- SVM's approach:
 - Define basis functions that are centred on the training data points
 - Select only a subset of these during training - *support vectors*
- Better, but this still typically increases with the size of the training set! :-(

Coping with the curse of dimensionality

- SVM's approach:
 - Define basis functions that are centred on the training data points
 - Select only a subset of these during training - *support vectors*
- Better, but this still typically increases with the size of the training set! :-)
- An alternative approach:
 - Fix the number of basis functions in advance
 - Allow them to be adaptive
- In other words: use parametric (non-linear) forms for the basis functions, in which the parameter values are adapted during training

Coping with the curse of dimensionality

- SVM's approach:
 - Define basis functions that are centred on the training data points
 - Select only a subset of these during training - *support vectors*
- Better, but this still typically increases with the size of the training set! :-)
- An alternative approach:
 - Fix the number of basis functions in advance
 - Allow them to be adaptive
- In other words: use parametric (non-linear) forms for the basis functions, in which the parameter values are adapted during training
- This is what neural networks do!

Linear models - another perspective

- Linear models considered so far took the form:

$$y = f(b + w_1x_1 + \cdots + w_Dx_D) = f\left(b + \sum_{i=1}^D w_ix_i\right)$$

where f is a non-linear function and $\{w_i\}$ are *weights* (coefficients)
and b is a *bias* term (intercept)

e.g. f is the sigmoid for logistic regression, or identity for linear regression

Linear models - another perspective

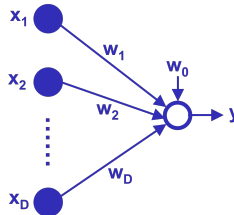
- Linear models considered so far took the form:

$$y = f(b + w_1x_1 + \cdots + w_Dx_D) = f\left(b + \sum_{i=1}^D w_ix_i\right)$$

where f is a non-linear function and $\{w_i\}$ are *weights* (coefficients) and b is a *bias* term (intercept)

e.g. f is the sigmoid for logistic regression, or identity for linear regression

- Another perspective



- The unshaded circle computes a weighted sum of all inputs and passes the result through a non-linearity $f(\dots)$

Adaptive parametric basis functions

- Linear models considered so far took the form:

$$y = f(b + w_1x_1 + \cdots + w_Dx_D) = f\left(b + \sum_{i=1}^D w_ix_i\right)$$

where f is a non-linear function and $\{w_i\}$ are *weights* and b is a *bias* term

Adaptive parametric basis functions

- Linear models considered so far took the form:

$$y = f(b + w_1x_1 + \cdots + w_Dx_D) = f\left(b + \sum_{i=1}^D w_ix_i\right)$$

where f is a non-linear function and $\{w_i\}$ are *weights* and b is a *bias* term

- With linear combinations of M **fixed** basis functions $\phi_m(\mathbf{x})$:

$$y = f\left(b + \sum_{m=1}^M w_m \phi_m(\mathbf{x})\right)$$

Adaptive parametric basis functions

- Linear models considered so far took the form:

$$y = f(b + w_1x_1 + \cdots + w_Dx_D) = f\left(b + \sum_{i=1}^D w_i x_i\right)$$

where f is a non-linear function and $\{w_i\}$ are *weights* and b is a *bias* term

- With linear combinations of M **fixed** basis functions $\phi_m(\mathbf{x})$:

$$y = f\left(b + \sum_{m=1}^M w_m \phi_m(\mathbf{x})\right)$$

- Make the basis functions $\phi_m(\mathbf{x})$ to be **non-linear parametric** functions:

$$y = f\left(b + \sum_{m=1}^M w_m g\left(b_m + \sum_{i=1}^D w_{m,i} x_i\right)\right)$$

where g is a non-linearity, e.g. sigmoid, tanh or ReLU

- Allow parameters of basis function $\{b_m\}$ and $\{w_{m,i}\}$ to be adjusted during training, along with the weights $\{w_m\}$ and bias b

Adaptive parametric basis functions - another perspective

- Make the basis functions $\phi_m(\mathbf{x})$ to be **non-linear parametric** functions:

$$y = f \left(b + \sum_{m=1}^M w_m g \left(b_m + \sum_{i=1}^D w_{m,i} x_i \right) \right)$$

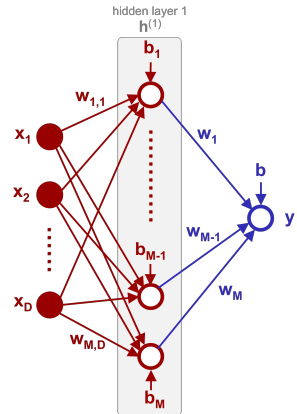
Adaptive parametric basis functions - another perspective

- Make the basis functions $\phi_m(\mathbf{x})$ to be **non-linear parametric** functions:

$$y = f\left(b + \sum_{m=1}^M w_m g\left(b_m + \sum_{i=1}^D w_{m,i} x_i\right)\right)$$

- An alternative perspective:
 - Hidden *layer* learns a new representation **h** of the original data **x**
 - In matrix form: $\mathbf{h} = g(\mathbf{b} + \mathbf{W}\mathbf{x})$
where $h_m = g(b_m + \sum_{i=1}^D w_{m,i} x_i)$
 - Vector **b** contains all *bias terms* $\{b_m\}$
 - **Output layer** is just a linear function of **h**

$$y = f\left(b + \sum_{m=1}^M w_m h_m\right)$$



Playtime!

- "6. Adaptive parametric basis functions.ipynb" notebook
- Estimated duration: 40 min

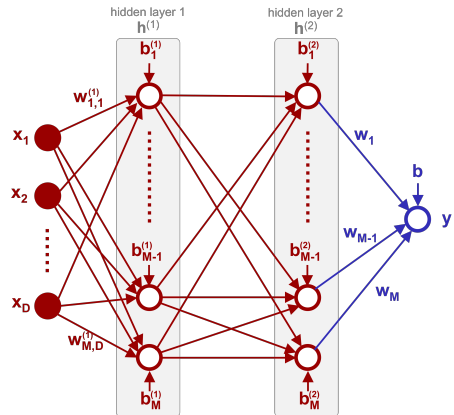
Stacking layers

- But why just 1 hidden layer? Why not build more complex networks?

Stacking layers

- But why just 1 hidden layer? Why not build more complex networks?
- Each layer computes a non-linear transformation $\mathbf{h}^{(l)}$ of the previous one

$$\mathbf{h}^{(l)} = g(\mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$$
- Each unshaded circle is called a *neuron*
- There is a weight associated with each connection between neurons
- Stacking multiple layers of neurons results in a *neural network*
- First layer is the *input layer*
- Last layer is the *output layer*



Quick Quiz — How many parameters?

A fully connected layer maps $m_{l-1} = 100$ to $m_l = 50$. How many trainable parameters (weights + biases)?

- ① 5,000
- ② 5,050
- ③ 10,000
- ④ 10,050

Quick Quiz — How many parameters?

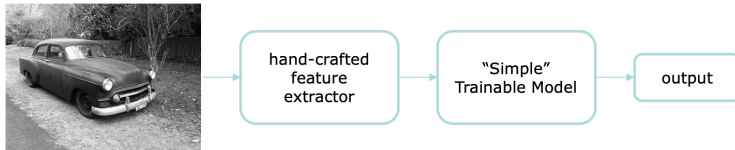
A fully connected layer maps $m_{l-1} = 100$ to $m_l = 50$. How many trainable parameters (weights + biases)?

- ① 5,000
- ② 5,050
- ③ 10,000
- ④ 10,050

Answer: $100 \times 50 + 50 = 5,050$.

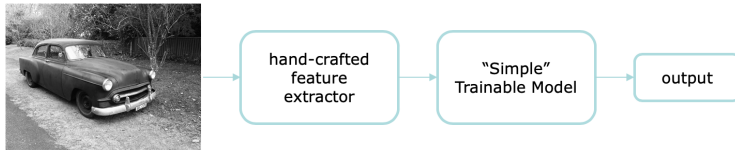
Learning complex feature representations

- Traditional pattern recognition models use hand-crafted features and a relatively simple trainable model



Learning complex feature representations

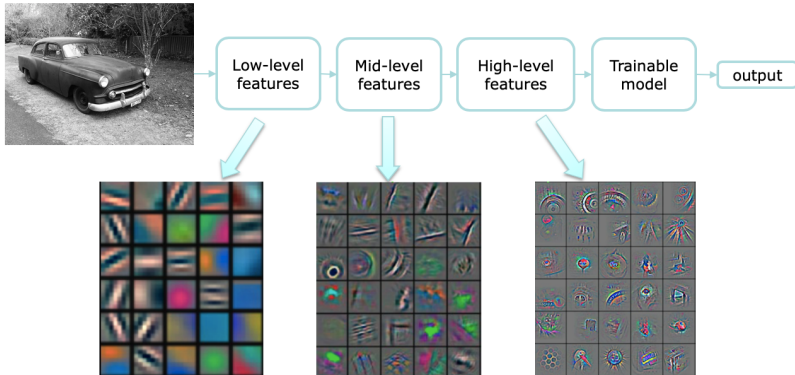
- Traditional pattern recognition models use hand-crafted features and a relatively simple trainable model



- Limitations:
 - Very tedious and costly to develop hand-crafted features
 - Hand-crafted features are usually highly dependent on one application, and cannot be transferred easily to others

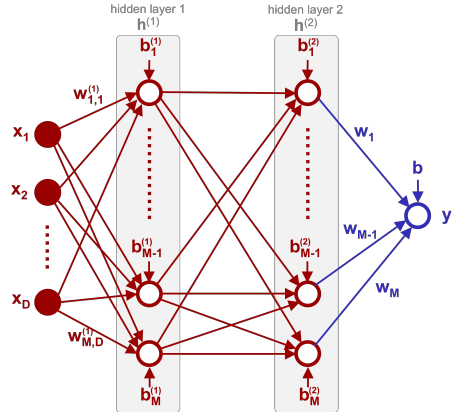
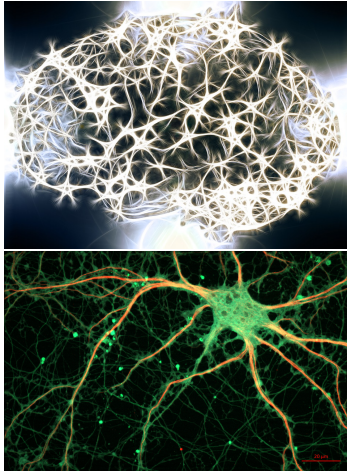
Learning complex feature representations

- Neural nets can learn rich hierarchical representations (i.e. features) automatically



- Hierarchy of representations with increasing level of abstraction
- Each stage is a kind of trainable nonlinear feature transform

Analogy with the human brain



By ZEISS Microscopy from Germany (Cultured Rat Hippocampal Neuron)
 CC BY 2.0 (<http://creativecommons.org/licenses/by/2.0>), via Wikimedia Commons
<https://pixabay.com/en/neurons-brain-cells-brain-structure-1739997/>

Activation functions - sigmoid

- Each layer computes a non-linear transformation $\mathbf{h}^{(l)}$ of the previous one

$$\mathbf{h}^{(l)} = g(\mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$$

- Which activation function g should we use?

Activation functions - sigmoid

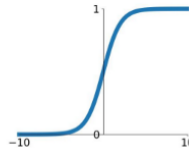
- Each layer computes a non-linear transformation $\mathbf{h}^{(l)}$ of the previous one

$$\mathbf{h}^{(l)} = g(\mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$$

- Which activation function g should we use?
- Sigmoid function

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



- Takes a real-valued number and “squashes” it into range between 0 and 1
- Nice interpretation as the firing rate of a neuron
- Sigmoid neurons saturate and kill gradients, thus NN will barely learn :-)

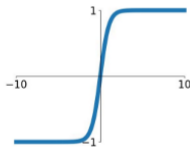
Activation functions - tanh

- Each layer computes a non-linear transformation $\mathbf{h}^{(l)}$ of the previous one

$$\mathbf{h}^{(l)} = g(\mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$$

- Which activation function g should we use?
- Hyperbolic tangent (tanh)

tanh
 $\tanh(x)$



- Takes a real-valued number and “squashes” it into range between -1 and 1
- Like sigmoid, tanh neurons saturate
- Unlike sigmoid, output is zero-centered

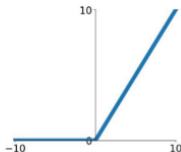
Activation functions - ReLU

- Each layer computes a non-linear transformation $\mathbf{h}^{(l)}$ of the previous one

$$\mathbf{h}^{(l)} = g(\mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$$

- Which activation function g should we use?
- Rectified linear unit (ReLU)

ReLU
 $\max(0, x)$



- Takes a real-valued number and thresholds it at zero
- Training is typically much faster
- Less expensive to compute
- Prevents the gradient vanishing problem

Activation cheat sheet

Name	Definition / Notes
ReLU	$f(x) = \max(0, x)$ (fast; sparse; watch “dying” units)
LeakyReLU	$f(x) = \max(\alpha x, x)$, $\alpha \approx 0.01$
ELU	$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$
SiLU / Swish	$f(x) = x \sigma(x)$, $\sigma(x) = \frac{1}{1 + e^{-x}}$
GELU	$f(x) = x \Phi(x) \approx \frac{1}{2}x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.0447x^3) \right] \right)$
tanh	$f(x) = \tanh(x)$
sigmoid	$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$
softplus	$f(x) = \ln(1 + e^x)$ (smooth ReLU alternative)
Property	Who / Why
Zero-centered	tanh, SiLU, GELU (often optimizes more smoothly)
Non-saturating	ReLU, LeakyReLU (fast learning; risk: dead units)
Smoothness	SiLU, GELU, ELU (gentler gradients; deeper nets)
Range	ReLU $[0, \infty)$; tanh $(-1, 1)$; sigmoid $(0, 1)$

$\Phi(x)$ denotes the standard normal CDF; $\sigma(x)$ the logistic function.

Neural nets for regression problems

- Target variables $y \in \mathbb{R}$
- Output layer with identity activation function

$$\hat{y} = g(b + \mathbf{w}^T \mathbf{h}^{(L)}) = b + \mathbf{w}^T \mathbf{h}^{(L)}$$

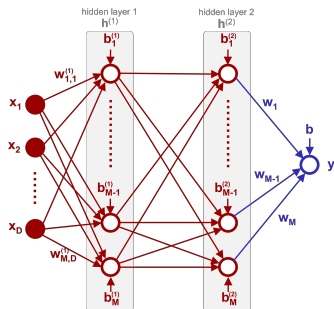
where $\mathbf{h}^{(L)}$ is the output of the last hidden layer

- Loss function: mean squared error (MSE)

$$L = \frac{1}{N} \sum_{j=1}^N (y_j - \hat{y}_j)^2$$

- Can model multiple target variables jointly by increasing number of neurons in output layer!

$$\hat{\mathbf{y}} = \mathbf{b} + \mathbf{W} \mathbf{h}^{(L)}$$



Neural nets for binary classification problems

- Target variables $y \in \{0, 1\}$
- Output layer with sigmoid activation function

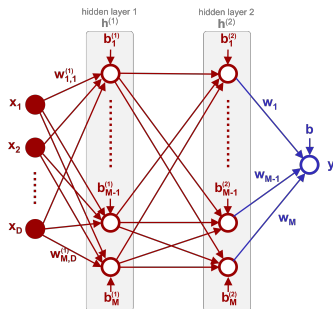
$$\hat{y} = p(y = 1) = \text{sigmoid}(b + \mathbf{w}^T \mathbf{h}^{(L)})$$

- Decision: predict class 1 if $\hat{y} > 0.5$, else class 0
- Loss function: binary cross-entropy loss

$$L = -\frac{1}{N} \sum_{j=1}^N y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)$$

- Can model multiple target variables jointly by increasing number of neurons in output layer!

$$\hat{\mathbf{y}} = \text{sigmoid}(\mathbf{b} + \mathbf{W} \mathbf{h}^{(L)})$$



Neural nets for multi-class classification problems

- Target variables $y \in \{1, \dots, C\}$ one-hot encoded!
- Output layer with softmax activation function

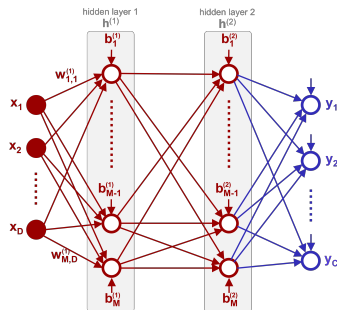
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{b} + \mathbf{W}\mathbf{h}^{(L)})$$

- Softmax generalizes sigmoid to multiple classes

$$\hat{y}_c = p(y = c) = \frac{\exp(b_c + \mathbf{w}_c^T \mathbf{h}^{(L)})}{\sum_{k=1}^C \exp(b_k + \mathbf{w}_k^T \mathbf{h}^{(L)})}$$

- Decision: predict class $c = \arg \max_c (\hat{y}_c)$
- Loss function: multi-class cross-entropy loss

$$L = -\frac{1}{N} \sum_{j=1}^N \sum_{c=1}^C y_{j,c} \log(\hat{y}_{j,c})$$



Activation cheat sheet — How to choose

Practical defaults:

- **Hidden layers (good first tries):**
 - ReLU (baseline)
 - LeakyReLU if many neurons are always 0
 - SiLU or GELU for strong smooth alternatives
- **Initialization:** He (Kaiming) for ReLU-family; Xavier/Glorot for tanh/sigmoid
- **Normalization:** BatchNorm/LayerNorm often stabilizes deep nets

Activation cheat sheet — How to choose

Practical defaults:

- **Hidden layers (good first tries):**
 - ReLU (baseline)
 - LeakyReLU if many neurons are always 0
 - SiLU or GELU for strong smooth alternatives
- **Initialization:** He (Kaiming) for ReLU-family; Xavier/Glorot for tanh/sigmoid
- **Normalization:** BatchNorm/LayerNorm often stabilizes deep nets

Match output to task:

- **Regression:** identity output; MSE/MAE loss
- **Binary class.:** σ (sigmoid); binary cross-entropy
- **Multi-class (single-label):** softmax; cross-entropy
- **Multi-label:** independent sigmoids; binary cross-entropy

Activation cheat sheet — How to choose

Practical defaults:

- **Hidden layers (good first tries):**
 - ReLU (baseline)
 - LeakyReLU if many neurons are always 0
 - SiLU or GELU for strong smooth alternatives
- **Initialization:** He (Kaiming) for ReLU-family; Xavier/Glorot for tanh/sigmoid
- **Normalization:** BatchNorm/LayerNorm often stabilizes deep nets

Match output to task:

- **Regression:** identity output; MSE/MAE loss
- **Binary class.:** σ (sigmoid); binary cross-entropy
- **Multi-class (single-label):** softmax; cross-entropy
- **Multi-label:** independent sigmoids; binary cross-entropy

Common pitfalls:

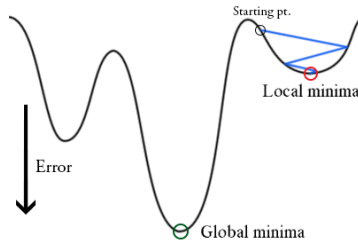
- *Dying ReLU:* switch to LeakyReLU or check initialization/learning rate
- *Saturation:* sigmoid/tanh near $\pm$ large $\Rightarrow$ vanishing gradients; prefer normalization
- *Mismatch:* do not use softmax for multi-label problems

Training

- Parameters $\{w\}$ are estimated to minimize prediction error according to some loss function L (e.g. mean squared error for regression problems)

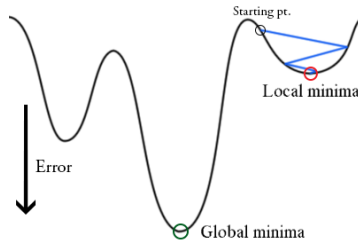
Training

- Parameters $\{w\}$ are estimated to minimize prediction error according to some loss function L (e.g. mean squared error for regression problems)
- Loss function is no longer convex!



Training

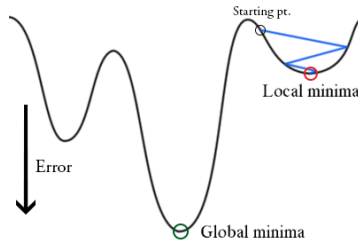
- Parameters $\{w\}$ are estimated to minimize prediction error according to some loss function L (e.g. mean squared error for regression problems)
- Loss function is no longer convex!



- Training based on (stochastic) gradient methods
 - Compute error gradients by taking derivatives of loss function w.r.t. weights
 - Backpropagate gradients through NNet by using the chain rule of derivatives
 - Take a step of size λ in the direction of the gradient

Training

- Parameters $\{w\}$ are estimated to minimize prediction error according to some loss function L (e.g. mean squared error for regression problems)
- Loss function is no longer convex!



- Training based on (stochastic) gradient methods
 - Compute error gradients by taking derivatives of loss function w.r.t. weights
 - Backpropagate gradients through NNet by using the chain rule of derivatives
 - Take a step of size λ in the direction of the gradient
- Convergence does not guarantee a global optimum!

Training in practice

- Use mini-batches of data
 - No need to use all observations in dataset to compute gradients
 - A small subsample of the observations to compute a fairly accurate (but noisy) gradient approximation

Training in practice

- Use mini-batches of data
 - No need to use all observations in dataset to compute gradients
 - A small subsample of the observations to compute a fairly accurate (but noisy) gradient approximation
 - Noisy gradients can actually be good for escaping local optima

Training in practice

- Use mini-batches of data
 - No need to use all observations in dataset to compute gradients
 - A small subsample of the observations to compute a fairly accurate (but noisy) gradient approximation
 - Noisy gradients can actually be good for escaping local optima
- Validation set and early stopping
 - Overfitting is a common problem in neural networks
 - Use validation set to detect when overfitting starts to occur
 - Stop training early when that happens



Training in practice

- Use mini-batches of data
 - No need to use all observations in dataset to compute gradients
 - A small subsample of the observations to compute a fairly accurate (but noisy) gradient approximation
 - Noisy gradients can actually be good for escaping local optima
- Validation set and early stopping
 - Overfitting is a common problem in neural networks
 - Use validation set to detect when overfitting starts to occur
 - Stop training early when that happens

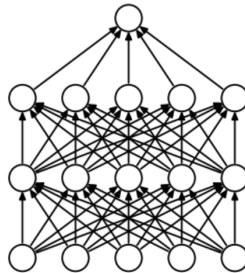


- Modern packages use automatic differentiation, adaptive step sizes, etc.

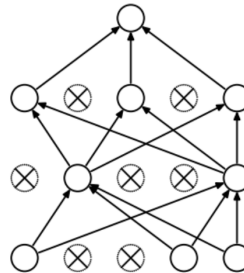
Dropout

Dropout

- Randomly “kill” a fraction of the neurons at each step during training



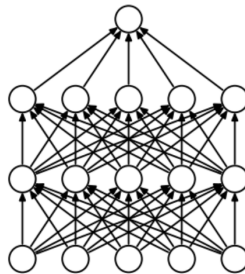
(a) Standard Neural Net



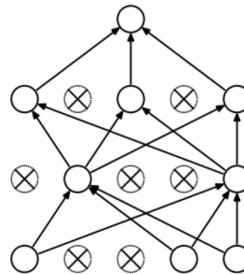
(b) After applying dropout.

Dropout

- Randomly “kill” a fraction of the neurons at each step during training



(a) Standard Neural Net



(b) After applying dropout.

- Number of nodes in the layer and its connectivity varies during training
- Each weight update is performed with a different “view” of the layer
- Somewhat analogous to training multiple different NNets in parallel
- Great technique for preventing overfitting!

Quick quiz — Dropout at inference/test time

True or False: Dropout should remain active during inference/test time.

Quick quiz — Dropout at inference/test time

True or False: Dropout should remain active during inference/test time.

Answer: False. Disable dropout (evaluation mode).

Advanced topics and trends (out of the scope of this course)



Advanced topics and trends (out of the scope of this course)



- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data

Advanced topics and trends (out of the scope of this course)



- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data

Advanced topics and trends (out of the scope of this course)



- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks

Advanced topics and trends (out of the scope of this course)

- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks
- Representation learning

Advanced topics and trends (out of the scope of this course)

- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks
- Representation learning
- Transfer learning and domain adaptation

Advanced topics and trends (out of the scope of this course)

- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks
- Representation learning
- Transfer learning and domain adaptation
- Multi-task learning

Advanced topics and trends (out of the scope of this course)

- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks
- Representation learning
- Transfer learning and domain adaptation
- Multi-task learning
- Adversarial training

Advanced topics and trends (out of the scope of this course)

- Other neural networks architectures
 - Convolutional neural networks (CNNs) - great for images and video
 - Recurrent neural network (RNNs) - great for sequential data
 - Auto-encoders - learn lower-dimensional representations of data
 - Graph CNNs - model graph data (e.g. social networks)
 - Seq2seq and encoder-decoder architectures - popular for machine translation
 - Generative adversarial networks (GANs) - good at generating “fake” data
- Uncertainty estimation and Bayesian neural networks
- Representation learning
- Transfer learning and domain adaptation
- Multi-task learning
- Adversarial training
- Deep Reinforcement Learning

Further learning

- Video series: “Deep learning without a Ph.D.”
<https://cloud.google.com/blog/products/gcp/learn-tensorflow-and-deep-learning-without-a-phd>
- Chapter 18 of “Data Science from scratch”
- “Deep learning”, Goodfellow, Bengio, Courville, 2016, Chapters 6-8
(<https://www-users.cs.umn.edu/~kumar/dmbook/ch4.pdf>)
- “Advanced Business Analytics” course at DTU
- “Deep learning” course at DTU

Playtime!

- "6. Neural Networks.ipynb" notebook
- Estimated duration: 90 min